

Agenda:

- Today, we will discuss the introduction to other SOTA such as Inception net and Resnet.
- And also how to use pretrained model to create embeddings for finding similarity.

✓ Problem statement:

Suppose you are a DataScientist in PINInterest company and your task is to return similar images on a given query image

✓ What is image similarity?

- Image similarity is the measure of how similar two images are.
- In other words, it quantifies the degree of similarity between intensity patterns in two images.
- Finding Image similarity also called reverse image search



✓ Download the Caltech101 dataset.

Description:

- Caltech-101 consists of pictures of objects belonging to 101 classes, plus one background clutter class.
- Each image is labelled with a single object. Each class contains roughly 40 to 800 images, totalling around 9k images.
- Images are of variable sizes, with typical edge lengths of 200-300 pixels.
- This version contains image-level labels only. The original dataset also contains bounding boxes.

Category Name	Sample Images		
Airplane			
Bonsai			
Camera			
Dalmatian			
Electric Guitar			
Euphonium			
Face			
Pizza			
Football			
Sunflower			

```
$ gdown https://drive.google.com/uc?id=137RyRjvTBkBiIfeYBNZBtViDHQ6_Ewsp --output caltech101.tar.gz
```

```
from google.colab import drive
drive.mount('/content/gdrive/',force_remount=True)
```

Mounted at /content/gdrive/

```
cd /content/gdrive/My Drive/Colab Notebooks/
```

[Errno 2] No such file or directory: '/content/gdrive/My Drive/Colab Notebooks/'
/content

```
!mkdir -p data
!pip install gdown
!gdown https://drive.google.com/uc?id=137RyRjvTBkBiIfeYBNZBtViDHQ6_Ewsp --output data/caltec
!tar -xvzf data/caltech101.tar.gz --directory data
!mv data/101_ObjectCategories data/caltech101
!rm -rf data/caltech101/BACKGROUND_Google
```

Streaming output truncated to the last 5000 lines.

```
101_ObjectCategories/chair/image_0005.jpg
101_ObjectCategories/chair/image_0006.jpg
101_ObjectCategories/chair/image_0007.jpg
101_ObjectCategories/chair/image_0008.jpg
101_ObjectCategories/chair/image_0010.jpg
101_ObjectCategories/chair/image_0011.jpg
101_ObjectCategories/chair/image_0012.jpg
101_ObjectCategories/chair/image_0013.jpg
101_ObjectCategories/chair/image_0014.jpg
101_ObjectCategories/chair/image_0016.jpg
101_ObjectCategories/chair/image_0017.jpg
101_ObjectCategories/chair/image_0018.jpg
```

101_ObjectCategories/chair/image_0019.jpg
101_ObjectCategories/chair/image_0020.jpg
101_ObjectCategories/chair/image_0022.jpg
101_ObjectCategories/chair/image_0023.jpg
101_ObjectCategories/chair/image_0024.jpg
101_ObjectCategories/chair/image_0025.jpg
101_ObjectCategories/chair/image_0026.jpg
101_ObjectCategories/chair/image_0028.jpg
101_ObjectCategories/chair/image_0029.jpg
101_ObjectCategories/chair/image_0030.jpg
101_ObjectCategories/chair/image_0031.jpg
101_ObjectCategories/chair/image_0032.jpg
101_ObjectCategories/chair/image_0034.jpg
101_ObjectCategories/chair/image_0035.jpg
101_ObjectCategories/chair/image_0036.jpg
101_ObjectCategories/chair/image_0037.jpg
101_ObjectCategories/chair/image_0038.jpg
101_ObjectCategories/chair/image_0040.jpg
101_ObjectCategories/chair/image_0041.jpg
101_ObjectCategories/chair/image_0042.jpg
101_ObjectCategories/chair/image_0043.jpg
101_ObjectCategories/chair/image_0044.jpg
101_ObjectCategories/chair/image_0046.jpg
101_ObjectCategories/chair/image_0047.jpg
101_ObjectCategories/chair/image_0048.jpg
101_ObjectCategories/chair/image_0049.jpg
101_ObjectCategories/chair/image_0050.jpg
101_ObjectCategories/chair/image_0052.jpg
101_ObjectCategories/chair/image_0053.jpg
101_ObjectCategories/chair/image_0054.jpg
101_ObjectCategories/chair/image_0055.jpg
101_ObjectCategories/chair/image_0056.jpg
101_ObjectCategories/chair/image_0058.jpg
101_ObjectCategories/chair/image_0059.jpg
101_ObjectCategories/chair/image_0060.jpg
101_ObjectCategories/chair/image_0061.jpg
101_ObjectCategories/chair/image_0062.jpg
101_ObjectCategories/chair/image_0001.jpg
101_ObjectCategories/chair/image_0002.jpg
101_ObjectCategories/chair/image_0003.jpg
101_ObjectCategories/chair/image_0009.jpg
101_ObjectCategories/chair/image_0015.jpg
101_ObjectCategories/chair/image_0021.jpg
101_ObjectCategories/chair/image_0027.jpg
101_ObjectCategories/chair/image_0033.jpg

✓ Importing Required libraries:

```
import numpy as np
from numpy.linalg import norm
import pickle
from tqdm import tqdm, tqdm_notebook
import os
```

```

import random
import time
import math
import tensorflow as tf
import numpy as np
from sklearn.manifold import TSNE
from sklearn.decomposition import PCA
import PIL
from PIL import Image
from sklearn.neighbors import NearestNeighbors
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras.preprocessing import image
from tensorflow.keras.applications.resnet50 import ResNet50, preprocess_input
import glob
import matplotlib
import matplotlib.pyplot as plt
import matplotlib.image as mpimg
%matplotlib inline

```

✓ Which Cnn pretrained architecture to use ?

- Recall from previous class, drawbacks of vgg & alexnet?
 - It is painfully slow to train.
 - The network architecture weights themselves are quite large (in terms of disk/bandwidth) becuz of FC layers in the end.

Can we have better than alexnet or vgg in terms of less parameters & better accuracy ?

Introduction to Resnet & Inception Net:

- Before solving the problem , lets try to understand much deeper state of the art CNN architectures compared to Alexnet and Vgg with less number of parameters and performed well on imagenet dataset.

```

def inception_module(layer_in, f1, f2_in, f2_out, f3_in, f3_out, f4_out):
    # 1x1 conv
    #Local Feature Extractor #Ant
    conv1 = Conv2D(f1, (1,1), padding='same', activation='relu')(layer_in)

    # 3x3 conv
    #!x1 reduce number of channels#Dog
    conv3 = Conv2D(f2_in, (1,1), padding='same', activation='relu')(layer_in) #Antman
    conv3 = Conv2D(f2_out, (3,3), padding='same', activation='relu')(conv3)

    # 5x5 conv#Elephant

```

```

conv5 = Conv2D(f3_in, (1,1), padding='same', activation='relu')(layer_in) #Antman
conv5 = Conv2D(f3_out, (5,5), padding='same', activation='relu')(conv5)

# 3x3 max pooling
pool = MaxPooling2D((3,3), strides=(1,1), padding='same')(layer_in)
pool = Conv2D(f4_out, (1,1), padding='same', activation='relu')(pool) #LFE
# concatenate filters, assumes filters/channels last
layer_out = concatenate([conv1#ant
                        , conv3,#dogs,cats
                        , conv5,#elephant
                        , pool], axis=1)

return layer_out

```

✓ Resnet

Code for identity block:

Moving on to the code, the code for the identity block is as shown below:

```

def identity_block(x, filter):
    # copy tensor to variable called x_skip
    F1,F2,F3 = filter
    x_skip = x

    # Layer 1 (Shrink)
    x = tf.keras.layers.Conv2D(F1, (1,1), padding = 'valid')(x)
    x = tf.keras.layers.BatchNormalization(axis=3)(x)
    x = tf.keras.layers.Activation('relu')(x)
    # Layer 2 (FE)
    x = tf.keras.layers.Conv2D(F2, (3,3), padding = 'same')(x)
    x = tf.keras.layers.BatchNormalization(axis=3)(x)
    # Layer 3 (Expansion)
    x = tf.keras.layers.Conv2D(F3, (1,1), padding = 'valid')(x)
    x = tf.keras.layers.BatchNormalization(axis=3)(x)

    # Add Residue (Direct addition)
    x = tf.keras.layers.Add()([x, x_skip])
    x = tf.keras.layers.Activation('relu')(x)
    return x

```

✓ Convolution block

```

def convolutional_block(x, s, filter):
    # copy tensor to variable called x_skip
    x_skip = x
    F1, F2, F3 = filter

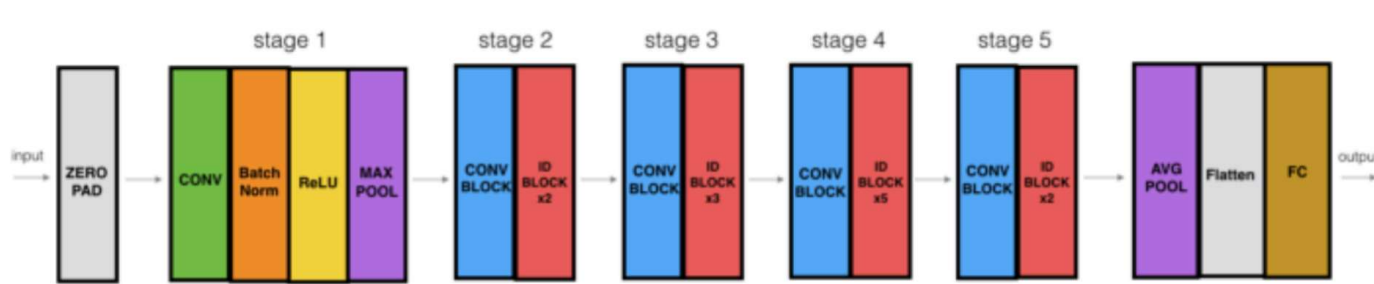
    # Layer 1
    x = tf.keras.layers.Conv2D(F1, (1,1), padding = 'valid', strides = (s,s))(x)
    x = tf.keras.layers.BatchNormalization(axis=3)(x)
    x = tf.keras.layers.Activation('relu')(x)
    # Layer 2
    x = tf.keras.layers.Conv2D(F2, (3,3), padding = 'same')(x)
    x = tf.keras.layers.BatchNormalization(axis=3)(x)
    x = tf.keras.layers.Activation('relu')(x)
    # Layer 3
    x = tf.keras.layers.Conv2D(F3, (1,1), padding = 'valid')(x)
    x = tf.keras.layers.BatchNormalization(axis=3)(x)

    # Processing Residue with conv(1,1)
    #Green CON 1x1 stride 2
    x_skip = tf.keras.layers.Conv2D(F3, (1,1), padding = 'valid', strides = (s,s))(x_skip)
    x_skip = tf.keras.layers.BatchNormalization(axis=3)(x_skip)

    # Add Residue
    x = tf.keras.layers.Add()([x, x_skip])
    x = tf.keras.layers.Activation('relu')(x)
    return x

```

✓ Resnet50 architecture:



```

def ResNet_50(input_shape=(224, 224, 3)):

    X_input = Input(input_shape)

    X = tf.keras.layers.ZeroPadding2D((3, 3))(X_input)

```

```
#stage 1 #Predefined
```

```
X = tf.keras.layers.Conv2D(64, (7, 7), strides=(2, 2), kernel_initializer=glorot_uniform
X = tf.keras.layers.BatchNormalization(axis=3)(X)
X = tf.keras.layers.Activation('relu')(X)
X = tf.keras.layers.MaxPooling2D((3, 3), strides=(2, 2))(X)
```

```
#stage 2
```

```
X = convolutional_block(X, s=1, filters=[64, 64, 256])
X = identity_block(X, [64, 64, 256])
X = identity_block(X, [64, 64, 256])
```

```
#stage 3
```

```
X = convolutional_block(X,s=2, filters=[128, 128, 512])
X = identity_block(X, [128, 128, 512])
X = identity_block(X, [128, 128, 512])
X = identity_block(X, [128, 128, 512])
```

```
#stage 4
```

```
X = convolutional_block(X,s=2, filters=[256, 256, 1024])
X = identity_block(X, [256, 256, 1024])
X = identity_block(X, [256, 256, 1024])
X = identity_block(X, [256, 256, 1024])
X = identity_block(X, [256, 256, 1024])
X = identity_block(X, [256, 256, 1024])
```

```
#stage 5
```

```
X = convolutional_block(X,s=2, filters=[512, 512, 2048])
X = identity_block(X, [512, 512, 2048])
X = identity_block(X, [512, 512, 2048])
```

```
X = tf.keras.layers.AveragePooling2D(pool_size=(2, 2), padding='same')(X)
```

```
model = Model(inputs=X_input, outputs=X, name='ResNet50')
```

```
return model
```

Lets back to our problem of deciding the pretrained cnn model-

- Since resnet50 performs better than googlenet in terms of accuracy and parameters.
- Hence we are going to use pretrained resnet50 model as our feature extractor for image similarity.

Quick Recall about Transfer Learning:

- Deep Convolutional Neural network from scratch takes days to train and its training requires lots of computational resources.
- So to overcome this we are using transfer learning in this Keras implementation of ResNet 50 .
- Here, we will reuse the model weights from pre-trained models that were developed for standard computer vision benchmark datasets like ImageNet.

✓ Loading the Resnet50 model: Pretrained

```
model = ResNet50(weights='imagenet',
                  include_top=False, #I dont need the final softmax layer
                  input_shape=(224, 224, 3),
                  pooling='avg'
                  )#Pretrained
```

```
model.summary()
```



Model: "resnet50"

Layer (type)	Output Shape	Param #	Connected to
=====			
input_3 (InputLayer)	[(None, 224, 224, 3)]	0	[]
conv1_pad (ZeroPadding2D)	(None, 230, 230, 3)	0	['input_3[0][0]']
conv1_conv (Conv2D)	(None, 112, 112, 64)	9472	['conv1_pad[0][0]']
conv1_bn (BatchNormalization)	(None, 112, 112, 64)	256	['conv1_conv[0][0]']
conv1_relu (Activation)	(None, 112, 112, 64)	0	['conv1_bn[0][0]']
pool1_pad (ZeroPadding2D)	(None, 114, 114, 64)	0	['conv1_relu[0][0]']
pool1_pool (MaxPooling2D)	(None, 56, 56, 64)	0	['pool1_pad[0][0]']
conv2_block1_1_conv (Conv2D)	(None, 56, 56, 64)	4160	['pool1_pool[0][0]']

conv2_block1_1_bn (BatchNormal ization)	(None, 56, 56, 64)	256	['conv2_block1_1_con
conv2_block1_1_relu (Activatio n)	(None, 56, 56, 64)	0	['conv2_block1_1_bn[
conv2_block1_2_conv (Conv2D)	(None, 56, 56, 64)	36928	['conv2_block1_1_rel
conv2_block1_2_bn (BatchNormal ization)	(None, 56, 56, 64)	256	['conv2_block1_2_con
conv2_block1_2_relu (Activatio n)	(None, 56, 56, 64)	0	['conv2_block1_2_bn[
conv2_block1_0_conv (Conv2D)	(None, 56, 56, 256)	16640	['pool1_pool[0][0]']
conv2_block1_3_conv (Conv2D)	(None, 56, 56, 256)	16640	['conv2_block1_2_rel
conv2_block1_0_bn (BatchNormal ization)	(None, 56, 56, 256)	1024	['conv2_block1_0_con
conv2_block1_3_bn (BatchNormal ization)	(None, 56, 56, 256)	1024	['conv2_block1_3_con
conv2_block1_add (Add)	(None, 56, 56, 256)	0	['conv2_block1_0_bn['conv2_block1_3_bn[
conv2_block1_out (Activation)	(None, 56, 56, 256)	0	['conv2_block1_add[0
conv2_block2_1_conv (Conv2D)	(None, 56, 56, 64)	16448	['conv2_block1_out[0

✓ Feature Extraction :

Let's define a function to extract image features given an image and a model.

```
def extract_features(img_path, model):
    input_shape = (224, 224, 3)
    img = image.load_img(img_path,
                          target_size=(input_shape[0], input_shape[1]))
    img_array = image.img_to_array(img)
    expanded_img_array = np.expand_dims(img_array, axis=0)
    preprocessed_img = preprocess_input(expanded_img_array)

    #Feature Extractor
    features = model.predict(preprocessed_img)

    flattened_features = features.flatten()
```

```

    normalized_features = flattened_features / norm(flattened_features)
    return normalized_features

def extract_features(img_path, model):
    input_shape = (224, 224, 3)
    img = image.load_img(img_path,
                          target_size=(input_shape[0], input_shape[1]))
    img_array = image.img_to_array(img)
    expanded_img_array = np.expand_dims(img_array, axis=0)
    preprocessed_img = preprocess_input(expanded_img_array)

    #Feature Extractor
    features = model.predict(preprocessed_img)

    #flattened_features = features.flatten()
    #normalized_features = flattened_features / norm(flattened_features)
    return normalized_features

```

Let's see the feature length the model generates by running on an example image. If you don't have the usual cat image available locally, let's download it!

```

try:
    import google.colab
    IS_COLAB_ENV = True
except:
    IS_COLAB_ENV = False

IMG_PATH = 'cat.jpg'
if IS_COLAB_ENV:
    !curl https://drive.google.com/file/d/1r0aOGw-SFH_lp8Sx8wRAksU79M8Wz8k4/view?usp=sharing -
    IMG_PATH = 'cat.jpg'

```

```

➡ % Total      % Received % Xferd  Average Speed   Time    Time       Time  Current
   100   661k   100   661k    0     0  1072k      0  --:--:--  --:--:--  --:--:--  1072k

```

```

features = extract_features(IMG_PATH, model)
print("Total length of features for one image: ", len(features))

```

```

➡ Total length of features for one image: 2048

```

- Now, we will see how much time it takes to extract features of one image.
- For the cat image, it should ideally be around 81 ms.
- Depending on your image size, the final time may be different and it depends on the time to read, resize, preprocess, and extract features.

```
%timeit features = extract_features('cat.jpg', model) #Embedding
```

➡ 10 loops, best of 5: 99.8 ms per loop

✓ Benchmarking time taken to extract features over the entire dataset

The time taken to extract features is dependent on a few factors such as image size, computing power etc.

A better benchmark would be running the network over an entire dataset.

A simple change to the existing code will allow this.

Let's make a handy function to recursively get all the image files under a root directory.

```
extensions = ['.jpg', '.JPG', '.jpeg', '.JPEG', '.png', '.PNG']
```

```
def get_file_list(root_dir):
    file_list = []
    for root, directories, filenames in os.walk(root_dir):
        for filename in filenames:
            if any(ext in filename for ext in extensions):
                filepath = os.path.join(root, filename)
                if os.path.exists(filepath):
                    file_list.append(filepath)
            else:
                print(filepath)
    return file_list
```

Now, let's run the extraction over the entire dataset and time it.

```
# path to the your datasets
root_dir = 'data/caltech101'
filenames = sorted(get_file_list(root_dir))
print(len(filenames))
```

➡ 8677

Now, let's run the extraction over the entire dataset and time it.

```
BATCH_SIZE = 128
generator = tf.keras.utils.image_dataset_from_directory(root_dir,
```

```

shuffle=False,
batch_size=BATCH_SIZE,
image_size=(224,224))

```

```

num_images = len(generator.file_paths)
num_epochs = int(math.ceil(num_images / BATCH_SIZE))

```

```

start_time = time.time()
feature_list = []
feature_list = model.predict(generator, num_epochs)
end_time = time.time()

```

```

➡ Found 8677 files belonging to 101 classes.

```

```

for i, features in enumerate(feature_list):
    feature_list[i] = features / norm(features)

feature_list = feature_list.reshape(len(feature_list), -1)

print("Num images    = ", len(generator.file_paths))
print("Shape of feature_list = ", feature_list.shape)
print("Time taken in min = ", (end_time - start_time)/60)

```

```

➡ Num images    = 8677
  Shape of feature_list = (8677, 2048)
  Time taken in min = 16.577463599046073

```

By now, we have generated features from the entire dataset of images.

With the benchmarking experiments squared away, let's save the features as intermediate files to use later.

```

pickle.dump(generator.class_names, open('./data/class_ids-caltech101.pickle',
                                         'wb'))
pickle.dump(generator.file_paths, open('./data/filenames-caltech101.pickle', 'wb'))
pickle.dump(
    feature_list,
    open('./data/features-caltech101-' + 'resnet50' + '.pickle', 'wb'))

```

That's all folks! We're done with the feature extraction part

✓ Similarity Search:

- Given a QUERY image, our aim is to find another photo in our dataset similar to the current one.

- We begin by loading the precomputed features:

```
filenames = pickle.load(open('./data/filenames-caltech101.pickle', 'rb'))
feature_list = pickle.load(open('./data/features-caltech101-resnet50.pickle',
                                'rb'))
class_ids = pickle.load(open('./data/class_ids-caltech101(1).pickle', 'rb'))
```

```
num_images = len(filenames)
num_features_per_image = len(feature_list[0])
print("Number of images = ", num_images)
print("Number of features per image = ", num_features_per_image)
```

```
⇒ Number of images = 8677
   Number of features per image = 2048
```

```
feature_list.shape
```

```
⇒ (8677, 2048)
```

```
len(class_ids)
```

```
⇒ 8677
```

✓ How we find similar images after feature extraction ?

Using Knearest Neighbors :

- We'll use Python's machine learning library scikit-learn for finding nearest neighbors of the query features; that is, features that represent a query image.
- We train a nearest-neighbor model using the brute-force algorithm to find the nearest five neighbors based on Euclidean distance

```
random_index = random.randint(0, num_images)
```

```
from sklearn.neighbors import NearestNeighbors
```

```
neighbors = NearestNeighbors(n_neighbors=5, algorithm='brute', metric='euclidean').fit(feature_list)
distances, indices = neighbors.kneighbors([feature_list[random_index]])
```

- Now you have both the indices and distances of the nearest five neighbors of the image at random_index.

- Notice the quick execution of the first step — the training step. Unlike training most machine learning models, which can take from several minutes to hours on large datasets,
 - instantiating the nearest-neighbor model is instantaneous because at training time there isn't much processing.
- This is also called lazy learning because all the processing is deferred to classification or inference time.

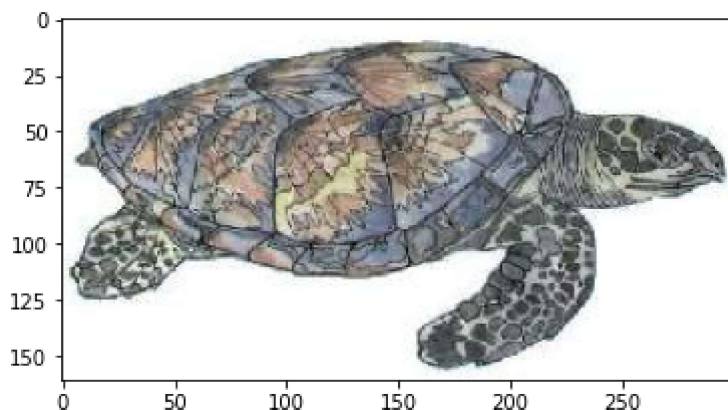
Now, let's look at the time it takes to search for the nearest neighbors for the selected random image using the trained model with the Brute force algorithm.

```
%timeit distances, indices = neighbors.kneighbors([feature_list[random_index]])
```

➞ 10 loops, best of 5: 59.6 ms per loop

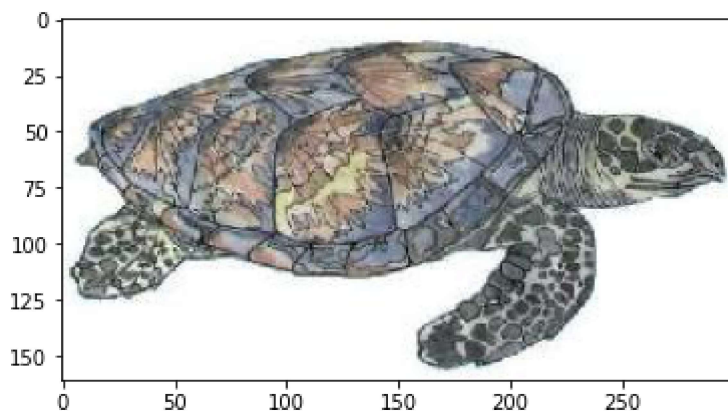
```
plt.imshow(mping.imread(filenamees[random_index]))
```

➞ <matplotlib.image.AxesImage at 0x7fc09f4a5310>



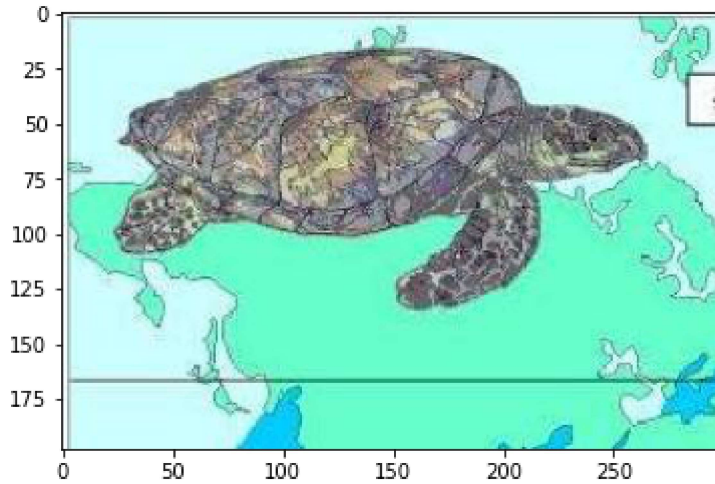
```
plt.imshow(mping.imread(filenamees[indices[0][0]]))
```

➞ <matplotlib.image.AxesImage at 0x7fc12018b790>



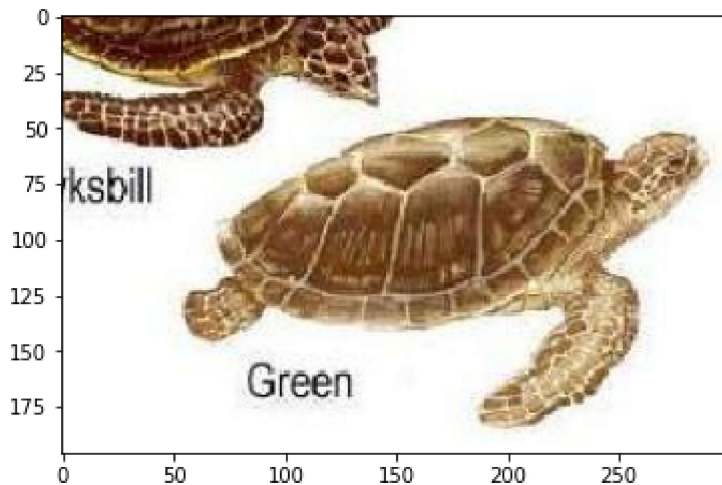
```
plt.imshow(mping.imread(filenamees[indices[0][1]]))
```

```
↪ <matplotlib.image.AxesImage at 0x7fc09e1b9e10>
```



```
plt.imshow(mping.imread(filenamees[indices[0][2]]))
```

```
↪ <matplotlib.image.AxesImage at 0x7fc09e1331d0>
```



```
for i in range(5):
    print(distances[0][i])
```

```
↪ 4.2146848e-08
0.61391467
0.6632041
0.6640254
0.6913976
```

```
# Helper function to get the classname
def classname(str):
    return str.split('/')[ -2]
```

```
# Helper function to get the classname and filename
```

```

def classname_filename(str):
    return str.split('/')[-2] + '/' + str.split('/')[-1]

# Helper functions to plot the nearest images given a query image
def plot_images(filenamees, distances):
    images = []
    for filename in filenamees:
        images.append(mpimg.imread(filename))
    plt.figure(figsize=(20, 10))
    columns = 4
    for i, image in enumerate(images):
        ax = plt.subplot(len(images) / columns + 1, columns, i + 1)
        if i == 0:
            ax.set_title("Query Image\n" + classname_filename(filenamees[i]))
        else:
            ax.set_title("Similar Image\n" + classname_filename(filenamees[i]) +
                          "\nDistance: " +
                          str(float("{0:.2f}".format(distances[i]))))
    plt.imshow(image)

for i in range(10):
    random_image_index = random.randint(0, 8000)
    distances, indices = neighbors.kneighbors(
        [feature_list[random_image_index]])
    # Don't take the first closest image as it will be the same image
    similar_image_paths = [filenamees[random_image_index]] + \
        [filenamees[indices[0][i]] for i in range(1, 4)]
    plot_images(similar_image_paths, distances[0])

# Calculating some stats
print("Median distance between all photos: ", np.median(distances))
print("Max distance between all photos: ", np.max(distances))
print("Median distance among most similar photos: ",
      np.median(distances[:, 2]))

```

 Median distance between all photos: 0.5878322
 Max distance between all photos: 0.5990474
 Median distance among most similar photos: 0.5878322

✓ Calculating Accuracy of the Brute Force Model :

```

# Helper function that calculates accuracy using the nearest neighbors brute force algorithm
def calculate_accuracy(feature_list):
    num_nearest_neighbors = 5
    correct_prediction = 0

```



```

incorrect_prediction = 0
neighbors = NearestNeighbors(n_neighbors=num_nearest_neighbors,
                             algorithm='brute',
                             metric='euclidean').fit(feature_list)

start = time.time()
for i in range(len(feature_list)):
    distances, indices = neighbors.kneighbors([feature_list[i]])
    for j in range(1, num_nearest_neighbors):
        if (classname(filenamees[i]) == classname(
            filenamees[indices[0][j]])):
            correct_prediction += 1
        else:
            incorrect_prediction += 1
end = time.time()
accuracy = round(
    100.0 * correct_prediction /
    (1.0 * correct_prediction + incorrect_prediction), 2), end - start
return accuracy

```

```

print("Accuracy on original feature set : ",
      calculate_accuracy(feature_list[:]))

```

➡ Accuracy on original feature set : (87.0, 584.7853474617004)

✓ Visualizing Image Clusters with t-SNE:

Let's step up the game by visualizing the entire dataset!

- To do this, we need to reduce the dimensions of the feature vectors because it's not possible to plot a 2,048-dimension vector (the feature-length) in two dimensions (the paper).
- The t-distributed stochastic neighbor embedding (t-SNE) algorithm reduces the high-dimensional feature vector to 2D, providing a bird's-eye view of the dataset, which is helpful in recognizing clusters and nearby images.
- t-SNE is difficult to scale to large datasets, so it is a good idea to reduce the dimensionality using Principal Component Analysis (PCA) and then call t-SNE:

```

# Perform PCA over the features
num_feature_dimensions=150 # Set the number of features
pca = PCA(n_components = num_feature_dimensions)
pca.fit(feature_list)
feature_list_compressed = pca.transform(feature_list)

```

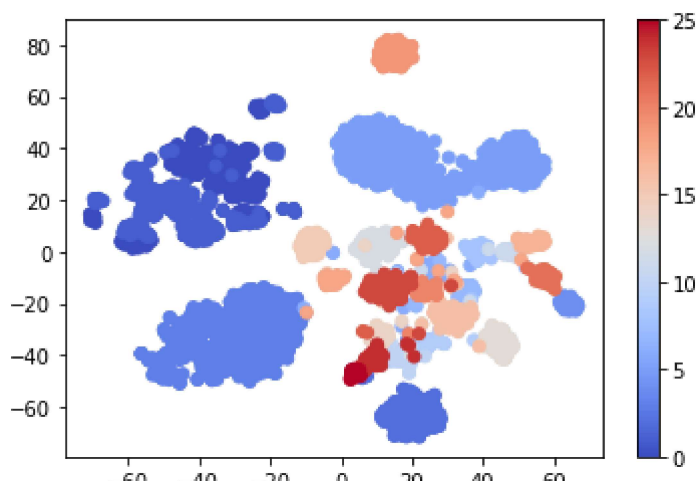
Time taken to search similar images for one image using PCA:

```
neighbors = NearestNeighbors(n_neighbors=5, algorithm='brute', metric='euclidean').fit(features)
distances, indices = neighbors.kneighbors([feature_list_compressed[random_index]])
%timeit distances, indices = neighbors.kneighbors([feature_list_compressed[random_index]])
```

➡ The slowest run took 6.26 times longer than the fastest. This could mean that an intermediate number of neighbors is best for your data. 100 loops, best of 5: 5.33 ms per loop

```
# For speed and clarity, we'll analyze about first half of the dataset.
selected_features = feature_list_compressed[:4000]
selected_class_ids = class_ids[:4000]
selected_filenames = filenames[:4000]
tsne_results = TSNE(n_components=2, verbose=1, metric='euclidean').fit_transform(selected_features)
# Plot a scatter plot from the generated t-SNE results
colormap = plt.cm.get_cmap('coolwarm')
scatter_plot = plt.scatter(tsne_results[:,0], tsne_results[:,1], c=selected_class_ids, cmap=colormap)
plt.colorbar(scatter_plot)
plt.show()
```

➡ /usr/local/lib/python3.7/dist-packages/sklearn/manifold/_t_sne.py:783: FutureWarning: The parameter 'perplexity' is deprecated in favor of 'n_neighbors'. Please use 'n_neighbors' instead.
FutureWarning,
/usr/local/lib/python3.7/dist-packages/sklearn/manifold/_t_sne.py:793: FutureWarning: The parameter 'early_exaggeration' is deprecated in favor of 'initial_neighbors_count'. Please use 'initial_neighbors_count' instead.
FutureWarning,
[t-SNE] Computing 91 nearest neighbors...
[t-SNE] Indexed 4000 samples in 0.004s...
[t-SNE] Computed neighbors for 4000 samples in 0.645s...
[t-SNE] Computed conditional probabilities for sample 1000 / 4000
[t-SNE] Computed conditional probabilities for sample 2000 / 4000
[t-SNE] Computed conditional probabilities for sample 3000 / 4000
[t-SNE] Computed conditional probabilities for sample 4000 / 4000
[t-SNE] Mean sigma: 0.147388
[t-SNE] KL divergence after 250 iterations with early exaggeration: 69.091812
[t-SNE] KL divergence after 1000 iterations: 1.111347

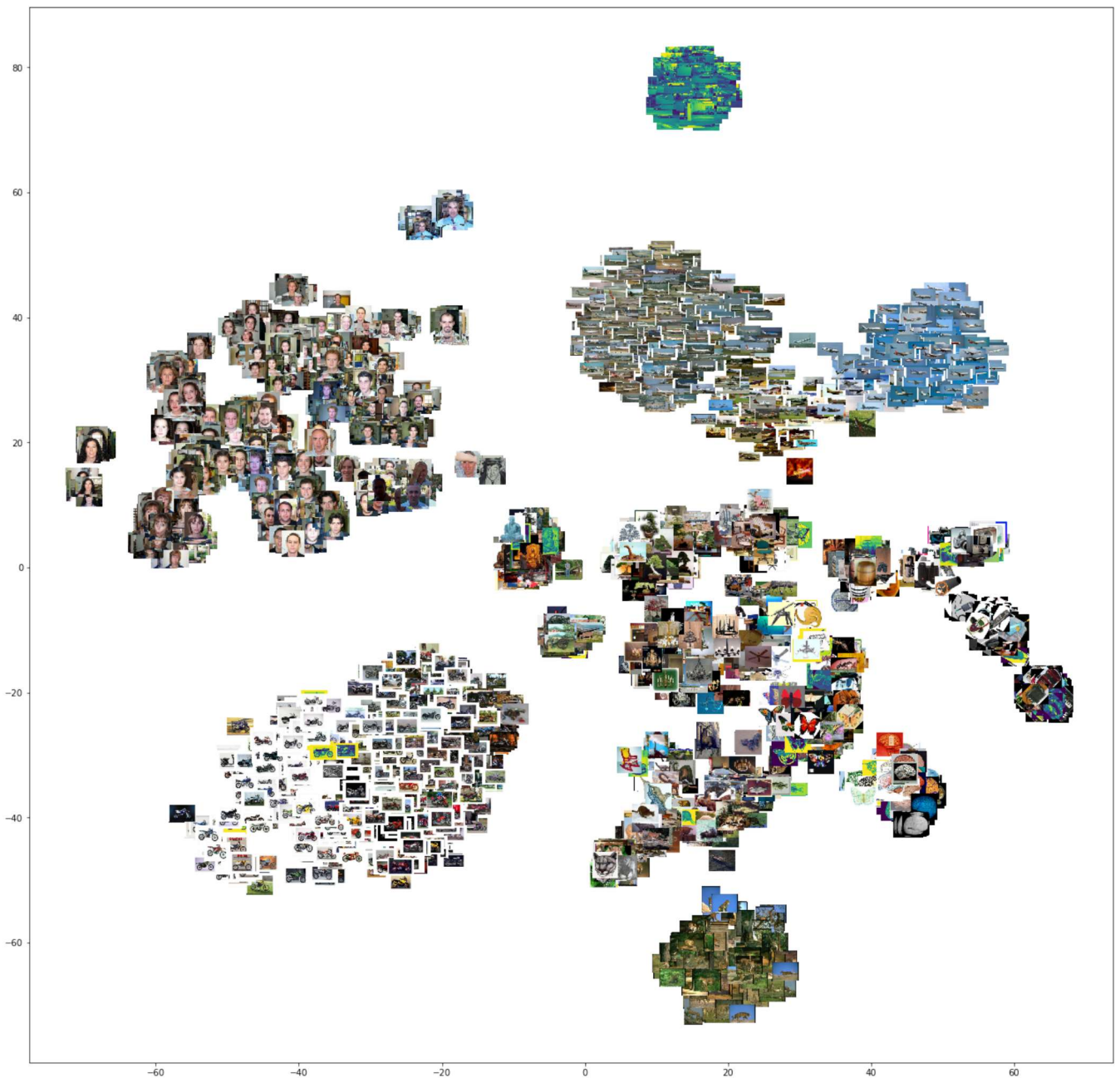


```
from matplotlib.offsetbox import OffsetImage, AnnotationBbox
from matplotlib.cbook import get_sample_data
```

```
def plot_images_in_2d(x, y, image_paths, axis=None, zoom=1):
    if axis is None:
        axis = plt.gca()
    x, y = np.atleast_1d(x, y)
    for x0, y0, image_path in zip(x, y, image_paths):
        image = Image.open(image_path)
        image.thumbnail((100, 100), Image.ANTIALIAS)
        img = OffsetImage(image, zoom=zoom)
        anno_box = AnnotationBbox(img, (x0, y0),
                                   xycoords='data',
                                   frameon=False)
        axis.add_artist(anno_box)
    axis.update_datalim(np.column_stack([x, y]))
    axis.autoscale()

def show_tsne(x, y, selected_filenames):
    fig, axis = plt.subplots()
    fig.set_size_inches(22, 22, forward=True)
    plot_images_in_2d(x, y, selected_filenames, zoom=0.3, axis=axis)
    plt.show()

show_tsne(tsne_results[:, 0], tsne_results[:, 1], selected_filenames)
```



Neat! There is a clearly demarcated cluster of human faces, flowers, vintage cars, ships, bikes, and a somewhat spread-out cluster of land and marine animals.

✓ Improving the Speed of Similarity Search:

- There are several opportunities to improve the speed of the similarity search step.
- For similarity search, we can make use of two strategies: either reduce the feature-length, or use a better algorithm to search among the features.
- Let's examine each of these strategies individually.

✓ Length of Feature Vectors:

- Ideally, we would expect that the smaller the amount of data in which to search, the faster the search should be.
- Recall that the ResNet-50 model gives 2,048 features.
- With each feature being a 32-bit floating-point, each image is represented by an 8 KB feature vector. For a million images, that equates to nearly 8 GB.
- Imagine how slow it would be to search among 8 GB worth of features.
- To give us a better picture of our scenario, Below table gives the feature-lengths that we get from different models:

Model	Bottleneck feature-length	Top-1% accuracy on ImageNet
VGG16	512	71.5%
VGG19	512	72.7%
MobileNet	1024	66.5%
InceptionV3	2048	78.8%
ResNet-50	2048	75.9%

- As we can see, almost all the models generate a large number of features.
- Imagine how much faster the search would be if we could reduce to a mere 100 features (a whopping reduction of 10 to 20 times!) without compromising the quality of the results.
- Apart from just the size, this is an even bigger improvement for big data scenarios, for which the data can be loaded into RAM all at once instead of periodically loading parts of it, thus

giving an even bigger speedup.

- PCA will help us make this happen.
- as you have seen above we have chosen 100 dimensions for PCA from the original 2048

Why did we pick 150 dimensions from the original 2,048?

Why not 200? PCA is representing our original feature vector but in reduced dimensions.

- Each new dimension has diminishing returns in representing the original vector (i.e., the new dimension might not explain the data much) and takes up valuable space.
- The number of dimensions in PCA is an important parameter that we can tune to the problem at hand.
- One way to directly justify a good threshold is to find a good balance between the number of features and its effect on accuracy versus speed:

✓ Accuracy of Brute Force over the PCA compressed Caltech101 features

```
pca_dimensions = [1, 2, 3, 4, 5, 10, 20, 50, 75, 100, 150, 200]
pca_accuracy = []
pca_time = []
```

```
for dimensions in pca_dimensions:
    pca = PCA(n_components=dimensions)
    pca.fit(feature_list)
    feature_list_compressed = pca.transform(feature_list[:])
    # Calculate accuracy over the compressed features
    accuracy, t = calculate_accuracy(feature_list_compressed[:])
    pca_time.append(t)
    pca_accuracy.append(accuracy)
    print("For PCA Dimensions = ", dimensions, ",\tAccuracy = ", accuracy, "%",
          ",\tTime = ", pca_time[-1])
```

```
➡ For PCA Dimensions = 1 ,      Accuracy = 15.43 % ,      Time = 5.8796515464782715
  For PCA Dimensions = 2 ,      Accuracy = 26.15 % ,      Time = 6.158180236816406
  For PCA Dimensions = 3 ,      Accuracy = 32.4 % ,      Time = 7.299118995666504
  For PCA Dimensions = 4 ,      Accuracy = 40.07 % ,      Time = 6.617396116256714
  For PCA Dimensions = 5 ,      Accuracy = 45.93 % ,      Time = 6.806637763977051
  For PCA Dimensions = 10 ,     Accuracy = 63.93 % ,      Time = 14.823916912078857
  For PCA Dimensions = 20 ,     Accuracy = 77.81 % ,      Time = 13.174192667007446
  For PCA Dimensions = 50 ,     Accuracy = 85.21 % ,      Time = 23.598203897476196
  For PCA Dimensions = 75 ,     Accuracy = 86.38 % ,      Time = 40.408857345581055
  For PCA Dimensions = 100 ,    Accuracy = 86.91 % ,      Time = 37.94637584686279
```

```
For PCA Dimensions = 150 ,      Accuracy = 87.03 % ,      Time = 51.85892105102539  
For PCA Dimensions = 200 ,      Accuracy = 87.05 % ,      Time = 63.817588806152344
```

- As is visible stats, there is little improvement in accuracy after increasing beyond a feature-length of 150 dimensions.
- With almost 20 times fewer dimensions (150) than the original (2,048), this offers drastically higher speed and less time on almost any search algorithm, while achieving similar (and sometimes slightly better) accuracy.
- Hence, 150 would be an ideal feature-length for this dataset.
- This also means that the first 150 dimensions contain the most information about the dataset.

How to scale it to millions of images ?

- Right now to find the similar images we are following the brute force approach
 - Although convenient to implement in two lines, it goes over each element and hence scales linearly with data size (number of items as well as the number of dimensions).
 - Having PCA take our feature vector from a length of 2,048 to 150 will not only yield a 20-times reduction in data size, but also result in an increase in speed of 20 times when using brute force. PCA does pay off!
 - Let's assume similarity searching a small collection of 10,000 images, now represented with 100 feature-length vectors, takes approximately 1 ms.
 - Even though this looks fast for 10,000 items, in a real production system with larger data, perhaps 10 million items, this will take more than a second to search.
 - Our system might not be able to fulfill more than one query per second per CPU core.
 - If you receive 100 requests per second from users, even running on multiple CPU cores of the machine (and loading the search index per thread), you would need multiple machines to be able to serve the traffic.
 - In other words, an inefficient algorithm means money, lots of money, spent on hardware.
 - Brute force is our baseline for every comparison.
- But Iterating over millions of images to find the most similar ones doesn't scale very well if we need to process all of the images every time we want to find a similar one.
- To optimize this process, we need to build some kind of index and find a way to iterate over it more efficiently.
- As in most algorithmic approaches, brute force is the slowest approach. Now that we have our baseline set, we will explore approximate nearest-neighbor algorithms.

Scaling Similarity Search with Approximate Nearest Neighbors

- Instead of guaranteeing the correct result as with the brute-force approach, approximation algorithms generally get the correct result because they are well, approximations.
- Most of the algorithms offer some form of tuning to balance between correctness and speed.
- It is possible to evaluate the quality of the results by comparing against the results of the brute-force baseline.
- Luckily Spotify, Google, and Facebook open-sourced their solution for exactly that problem.
- But there are also managed services if you don't want to take care of the required infrastructure and scaling on your own.
 - Open Source Frameworks like [Facebook's Faiss](#) and [Spotify Annoy](#), [Googles ScaNN](#)
 - Services like [Google Vertex AI Matching Engine](#), [IO Similarity Search](#), [Pinecone](#), [Milvus](#)

✓ PCA + Annoy:

- Annoy (Approximate Nearest Neighbors Oh Yeah) is a C++ library with Python bindings for searching nearest neighbors.
- Synonymous with speed, it was released by Spotify and is used in production to serve their music recommendations.
- To use annoy, install it using pip: `pip3 install annoy`

`!pip3 install Annoy`

```

🔗 Looking in indexes: https://pypi.org/simple, https://us-python.pkg.dev/colab-wheels/public
Collecting Annoy
  Downloading annoy-1.17.0.tar.gz (646 kB)
    |████████████████████████████████████████| 646 kB 28.9 MB/s
Building wheels for collected packages: Annoy
  Building wheel for Annoy (setup.py) ... done
  Created wheel for Annoy: filename=annoy-1.17.0-cp37-cp37m-linux_x86_64.whl size=394608
  Stored in directory: /root/.cache/pip/wheels/4f/e8/1e/7cc9ebbf8a87a3b9f8ba79408d4d31831
Successfully built Annoy
Installing collected packages: Annoy
Successfully installed Annoy-1.17.0

```

```
from annoy import AnnoyIndex
```

First, we build a search index with two hyperparameters - the number of dimensions of the dataset, and the number of trees.


```
# Time the indexing for Annoy
t = AnnoyIndex(150) # Length of item vector that will be indexed
starttime = time.time()
for i in range(num_images):
    feature = feature_list_compressed[i]
    t.add_item(i, feature)
endtime = time.time()
print(endtime - starttime)
t.build(40) # 40 trees
t.save('data/caltech101index.ann')
```

⚡ /usr/local/lib/python3.7/dist-packages/ipykernel_launcher.py:2: FutureWarning: The default value of 'allow_pickle' is deprecated in 1.1.0 and will be removed in 1.2.0. Please use 'allow_pickle=False' to avoid this warning.

0.3510735034942627
True

Annoy on one image: Now let's find out the time it takes to search the 5 nearest neighbors of one image.

```
random_image_index = 1001
```

```
u = AnnoyIndex(150)
%timeit u.get_nns_by_vector(feature_list_compressed[random_image_index], 5, include_distances
indexes = u.get_nns_by_vector(feature_list_compressed[random_image_index],
                             5,
                             include_distances=True)
```

⚡ /usr/local/lib/python3.7/dist-packages/ipykernel_launcher.py:1: FutureWarning: The default value of 'allow_pickle' is deprecated in 1.1.0 and will be removed in 1.2.0. Please use 'allow_pickle=False' to avoid this warning.

"""Entry point for launching an IPython kernel.
The slowest run took 100.98 times longer than the fastest. This could mean that an integer
100000 loops, best of 5: 19.9 µs per loop

- Now **THAT is blazing fast!** To put this in perspective, for such a modestly sized dataset, this can serve almost 15000 requests on a single CPU core.
- Considering most CPUs have multiple cores, it should be able to handle 100K+ requests on a single system. The best part is that it lets you share the same index in memory between multiple processes. Hence, the biggest index can be equivalent to the size of your overall RAM, making it possible to serve multiple requests on a single system.

Helper function to time the search for multiple images for Annoy.

Perform the search for the same image multiple times to get an average value.

```
def calculate_annoy_time():  
    for i in range(0, 100):  
        indexes = u.get_nns_by_vector(feature_list_compressed[random_image_index],  
                                     5,  
                                     include_distances=True)
```

Annoy on a set of images:

Time the search for multiple images for Annoy.

```
%time calculate_annoy_time()
```



```
CPU times: user 5.42 ms, sys: 0 ns, total: 5.42 ms  
Wall time: 4.56 ms
```